[Dat Nguyen]- Linux OS Overview & Kernel Module/Driver Basics

1. Basic concept of an Operating System

1.1 What is an Operating System?

An Operating System (OS) is system software that manages computer hardware and software resources and provides an environment for applications to run.

Examples of operating systems:

```
Windows: A popular operating system for personal computers.

Linux: Used on servers, embedded systems, and many IoT devices.

macOS: The operating system for Apple computers.

Android: The operating system for phones and embedded devices.

iOS: The operating system for iPhones.
```

Where is the OS located in a computer system?

```
+-------------------------------------+
|          User Applications          |
|     Browser, Editor, Games, etc.    |
+-------------------------------------+
|        System Call Interface        |
+-------------------------------------+
|          Operating System           |
```

```
|  +-------------------------------+  |
|  | Process Management            |  |
|  | Memory Management             |  |
|  | File System                   |  |
|  | Device Drivers                |  |
|  | Networking                    |  |
|  +-------------------------------+  |
+--------------------------------------+
|              Hardware                |
|   CPU | RAM | SSD | NIC | Devices    |
+--------------------------------------+
```

1.2 Why is an Operating System needed?

Without an operating system, using a computer to run regular applications would be much more complicated.

There are four main reasons why an operating system is necessary.

- 1: Hardware Abstraction
- 2: Resource Management

A computer has many finite resources:

```
CPU

RAM

Storage

Network interfaces

Peripheral devices
```

Meanwhile, multiple programs can run simultaneously and request to use these resources.

CPU Management:

On a system with a single CPU core, multiple processes can take turns using the CPU through scheduling and context switching mechanisms.

On a multi-core system, multiple processes or threads can also execute in parallel.

- 3: Protection and Security

A regular user is not allowed to arbitrarily modify all system files.

A process cannot arbitrarily read or write to the memory of another process.

- 4: Providing Common Services

Service	Function	Linux Example
Process management	Create and manage processes	`fork()`, `execve()`, `waitpid()`
Thread management	Support multi-threaded execution	`pthread_create()`
Memory management	Allocate and manage memory	`mmap()`, `brk()`
File management	Access and manage files	`open()`, `read()`, `write()`
IPC	Inter-process communication	Pipe, shared memory, message queues
Networking	Communication over the network	`socket()`, `bind()`, `connect()`
Synchronization	Synchronize activities	Mutex, semaphore, futex

alt text

2. Linux Operating System concept and its main components

2.1 System concept

2.2 Main components

Process Scheduling: Linux is a preemptive multitasking operating system. The kernel decides which process gets to use the CPU, for how long, and when it must relinquish it.

Memory Management: Utilizes Virtual Memory with two major advantages:

- Process Isolation: A process cannot read or write to the memory region of another process or the kernel.
- RAM Efficiency: Only a portion of a process needs to be kept in RAM at any given time.

File System Provision: Manages the creation, reading, writing, and deletion of files on disk.

Process Lifecycle: Loads programs into memory, allocates resources, and reclaims resources when processes terminate.

Device Access: Provides a standardized interface (via Device Drivers) for programs to communicate with mice, keyboards, monitors, hard drives, etc., without needing to know complex hardware details.

Networking: Transmits and receives network packets, routing packets on behalf of processes.

System Call API Provision: Acts as the primary gateway or interface allowing processes in user-space to request the kernel to perform privileged tasks.

3. User space vs. Kernel space: concept, memory, and process differences between the two

3.1 Concept

User space: the environment where regular applications run and utilize system resources through the interfaces provided by the operating system.

Kernel space: the address space reserved for the kernel and components running with kernel privileges.

- Why does Linux need such a separation?

1: System Protection

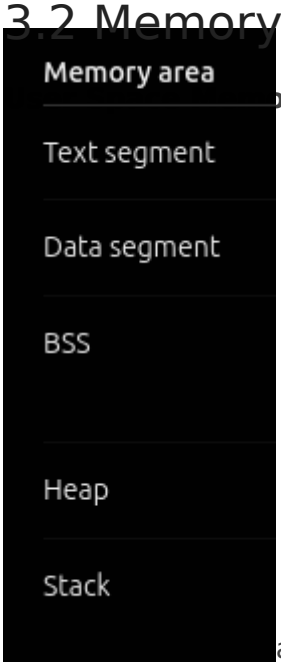
If all programs had permission to access kernel memory and control hardware, a bug in an application could severely impact the entire system.

=> The separation of privilege levels helps restrict dangerous operations.

2: Controlled Access

For example, a regular program can call open() to request opening a file. However, whether the file opens successfully or not depends on the path, access permissions, system state, and other conditions.

3.2 Memory



alt text

Kernel Space Memory

The kernel needs to store:

```
Process management information.

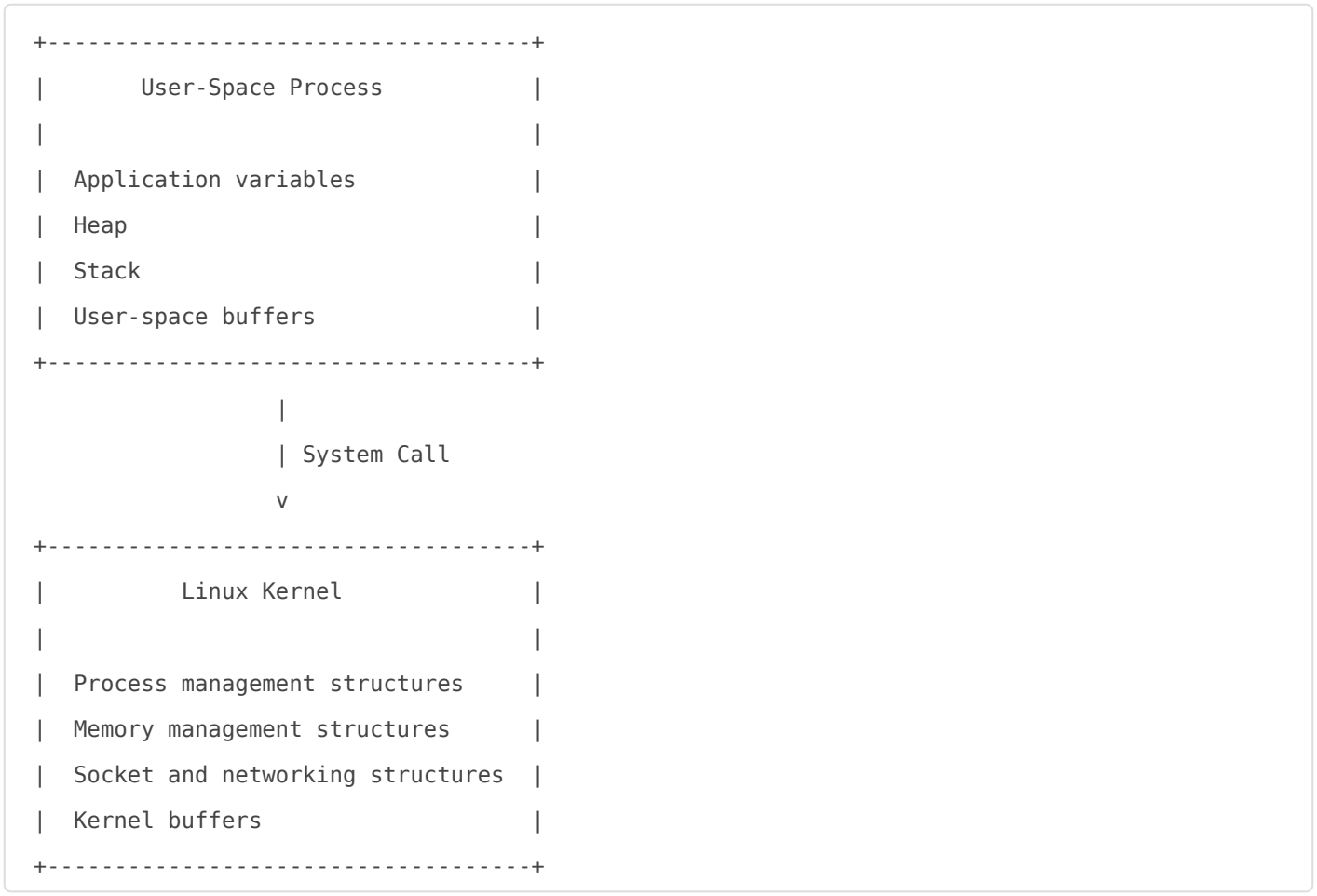
Filesystem data structures.

Socket and network buffer information.

Memory management structures.

Driver data and states.
```

Note: Kernel data structures are managed within the kernel environment, not as regular variables in a user-space process



3.3 Process

Criteria	User Space View	Kernel Space View
Nature of a Process	A running program that has its own memory space and thinks it owns the whole computer.	An object managed through a data structure called `task_struct`, which contains the PID, state, register values, permissions, open files, and more.
Interaction	Independent: A process cannot directly know what other processes are doing. It needs the kernel to create a new process or exit.	Full control: The kernel manages all processes. It knows which processes are running, ready to run, waiting for I/O, or sleeping.
Context Switching	A process cannot decide when to stop running and let another process use the CPU.	The kernel uses the **scheduler** to switch between processes. It saves the old process's CPU register values and loads the new process's register values.

3.4 Compare User Space and Kernel Space

Criterion	User space	Kernel space

Purpose	Running applications	Resource management and system service provision
CPU privilege	Typically user mode	Kernel mode
Memory	Process address space	Kernel address space and kernel-managed regions
System call	Sends service requests	Receives and processes requests
Hardware access	Typically via authorized interfaces	Can use kernel mechanisms to manage hardware
Execution errors	Usually affects the process	Can affect the entire system
Examples	TCP server, shell, C applications	Scheduler, filesystem code, kernel modules

4. Linux Kernel Components overview: how Memory Management, File System, Process Management, Network, and Device Driver subsystems relate to each other (conceptual survey, not deep-dive)

Process management:

- Process and thread management.
- CPU scheduling.
- Synchronization (mutex, semaphore, etc.).
- Signal and IPC management (pipe, message queue, etc.).

Memory Management:

- Physical and virtual memory management.
- Memory allocation and deallocation.
- Virtual memory (paging, swapping).
- Memory protection between processes.

File system:

- The kernel provides a uniform interface. Whether you are reading or writing data on any hard drive, system calls (such as open, read, write, close) are used in the exact same way. Thanks to this, application developers do not need to write separate code for each type of hard drive.

- Permissions & Security (Allowing (r/w) and denying permissions (---))

Device driver:

- Interacting with hardware: Working directly with the registers, I/O ports, or memory of hardware devices.
- Handling interrupts: Receiving and processing interrupt signals sent from hardware to the CPU when a device finishes a task or has new data.
- Providing APIs for other components: Offering programming interfaces (APIs) so that upper layers or other components in the kernel can call and use the device.

Network:

- Managing network communication (TCP/IP, UDP, ...)
- Socket API: Providing a programming interface so that applications can send and receive data over the network.
- Firewall: Built directly into the kernel to perform secure packet filtering.

5. Linux Kernel Module (LKM) concept: what it is and why modules exist; built-in vs. loadable modules

5.1 what is LKM

Linux Kernel Module (LKM): is a piece of code that can be loaded into a running kernel to add functionality to Linux without necessarily recompiling the entire kernel or rebooting the system.

5.2 Why does Linux need Kernel Modules?

If all features had to be statically built into the kernel, the operating system would face several limitations.

=> Not every system requires all features.

5.3 built-in vs. loadable modules

Built-in modules (=y): Integrated into the kernel during compilation.

Advantages:

- The functionality is available as soon as the kernel boots.
- There is no need to load a separate module file for that feature.

- It is useful for essential components required for the system to boot.

Disadvantages:

- Image Bloat: Because all source code and drivers are compiled directly into the kernel image file (vmlinuz), the kernel size becomes very bulky if too many features are integrated from the beginning.
- Permanent RAM Consumption: Every built-in component is loaded into memory and continuously occupies RAM for the entire duration the system is running, even if that hardware device is never connected or used.
- Inflexibility (Reboot Required for Changes): If the hardware changes or you want to update a new driver, you are forced to reconfigure, recompile the entire kernel, and reboot the system for the changes to take effect.

=> That is why modern systems use .ko files to load only what is needed into RAM, saving memory and making things much more flexible.

loadable modules (=m):

Code is compiled into a separate module and can be loaded or unloaded while the system is running, provided the necessary conditions are met (Kernel Version Matching, Root Privileges)

Advantages:

- Can be loaded when needed.
- Can be unloaded when no longer in use, provided the module supports unloading and is not currently in use.

Disadvantages:

- The module must be compatible with the currently running kernel.
- It may depend on other modules.
- A module that is currently in use or does not support unloading may not be removable.

```
        Module source code
              |
              v
      Build against kernel
         build environment
              |
              v
          Module file
             .ko
              |
              v
```

```
       Load into kernel
              |
              v
     Module initialization
              |
              v
       Module functionality
              |
              v
      Module cleanup / unload
```

Compare build_in and loadable module

Criterion	User space	Kernel space
Purpose	Running applications	Resource management and system service provision
CPU privilege	Typically user mode	Kernel mode
Memory	Process address space	Kernel address space and kernel-managed regions
System call	Sends service requests	Receives and processes requests
Hardware access	Typically via authorized interfaces	Can use kernel mechanisms to manage hardware
Execution errors	Usually affects the process	Can affect the entire system
Examples	TCP server, shell, C applications	Scheduler, filesystem code, kernel modules

6. Linux Driver Concept: How a Driver Fits Between Hardware and the Kernel's User-Facing Interfaces

What is Linux Driver?

A Linux device driver is kernel software responsible for managing and controlling a type of hardware device, while also providing an interface for other system components to use that device.

How does a driver communicate with hardware?

- Driver -> Hardware (The driver actively issues commands):

The driver reads or writes data directly to memory addresses or registers pre-assigned by the hardware manufacturer.

Register	Function
`CONTROL`	Control the device
`STATUS`	Report the device status
`DATA`	Exchange data

The driver can write to the control register to command the device to perform an operation, or read the status register to check whether the device has finished or not.

- Hardware -> Driver (The hardware reports events):

Interrupt (IRQ): Instead of forcing the driver to constantly wait and check if the hardware has finished its job (which wastes a lot of resources), the hardware does its work. Whenever an important event occurs (for example, a network packet has just arrived, or the user has just pressed a key), the hardware sends an electrical interrupt signal to "call" the CPU and the driver to immediately pause what they are doing and handle that data.

- Large data transfer: DMA (Direct Memory Access):

When transferring a large amount of data (such as reading/writing a hard drive or streaming a camera), if the CPU and driver are forced to copy byte by byte, it is very slow. A separate microchip called a DMA Controller is used. The hardware automatically copies data straight into RAM, and when finished, it sends an interrupt signal to notify the driver that the data retrieval is complete.

How applications communicate with drivers through an interface

When you write a program in the user space that wants to read data from a device, you call familiar functions such as:

```
open()

read()

write()

close()

ioctl() (used to configure the device)
```

However, these applications do not know and are not allowed to directly call the hardware or touch kernel memory.

The System Call bridge: When an application calls read(), the CPU switches from User Mode to Kernel Mode via the System Call mechanism.

7. Module lifecycle: init/exit functions, insmod/rmmod/modprobe, module parameters, dependencies

First, let's distinguish between two concepts:

Linux kernel: The core part of the operating system that manages processes, memory, devices, networking, and system resources.

Linux Kernel Module (LKM): A piece of code that can be dynamically loaded into a running kernel to add functionality, such as a driver for a device.

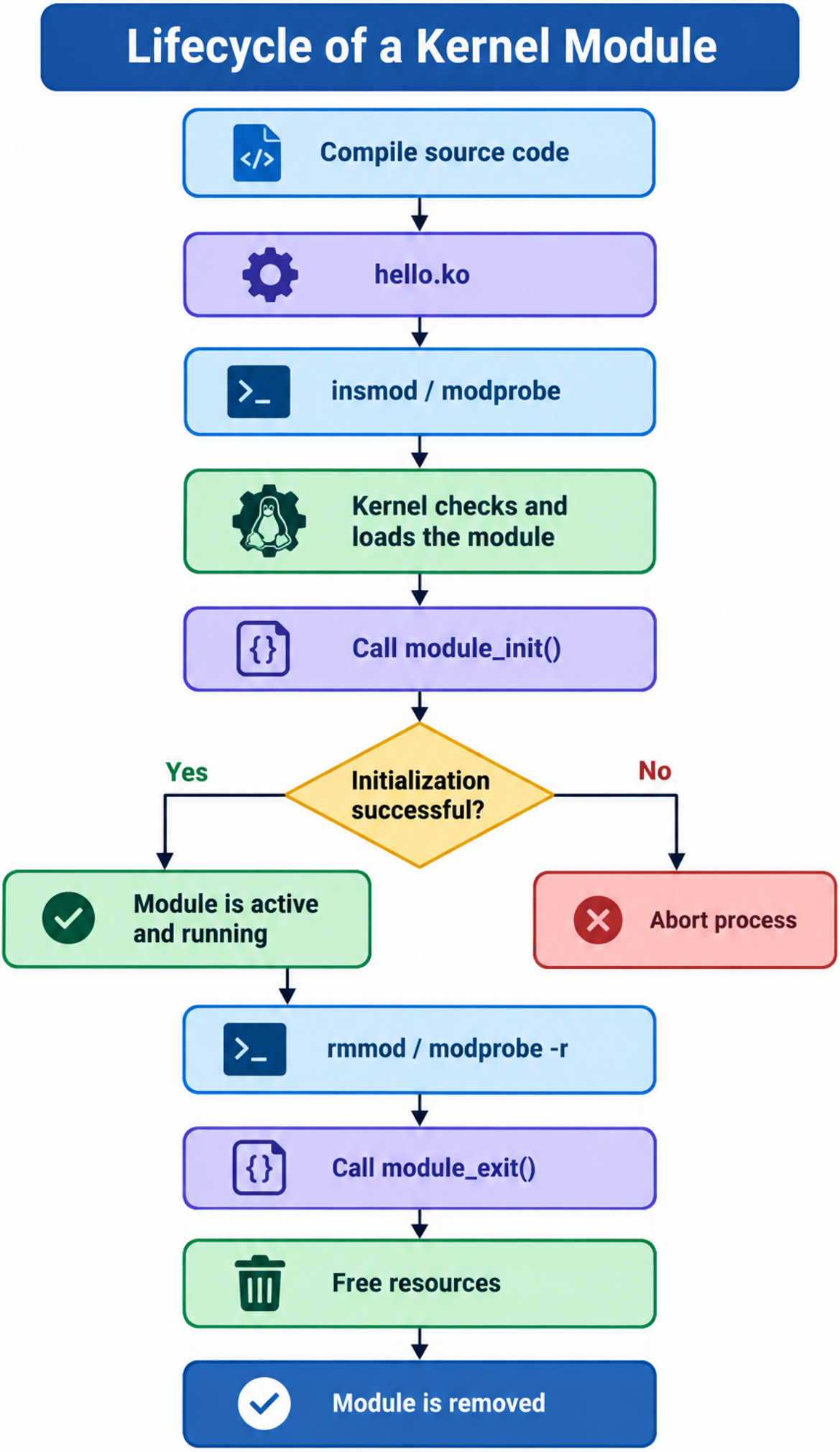
Normally, kernel code can be built in two ways:

- Method 1 (Built-in): The code is compiled directly into the kernel. When the kernel boots up, that functionality is already available.
- Method 2 (Loadable Kernel Module): The code is compiled into a module file, usually with the .ko (kernel object) extension, and then loaded into the kernel when needed.

Note: Loaded modules run with kernel privileges. Therefore, a serious bug in a module can freeze the system or cause a kernel panic.

Module lifecycle

Lifecycle of a Kernel Module



7.1 init/exit functions

Initialization function (module_init):

This is the first function that runs when the module is successfully loaded into the kernel.

- Tasks: Allocate resources, register the device with the kernel, request memory, or set up initial hardware.
- If this function returns an error value (not zero), the kernel will immediately refuse to load the module.

Cleanup function (module_exit):

- Runs right before the module is removed from the kernel.
- Tasks: Do the opposite of the initialization function — free allocated memory, unregister the device, and stop background processes to prevent memory leaks.

7.2 insmod/rmmod/modprobe

insmod (Insert Module):

- The most basic command to load a specific .ko file into the kernel.
- Disadvantage: It only loads the specified file and does not automatically resolve dependencies. If your module requires another module that is missing, insmod will throw an error.

rmmod (Remove Module):

- A command used to remove a running module from the kernel (calls the exit function and removes the code from RAM).

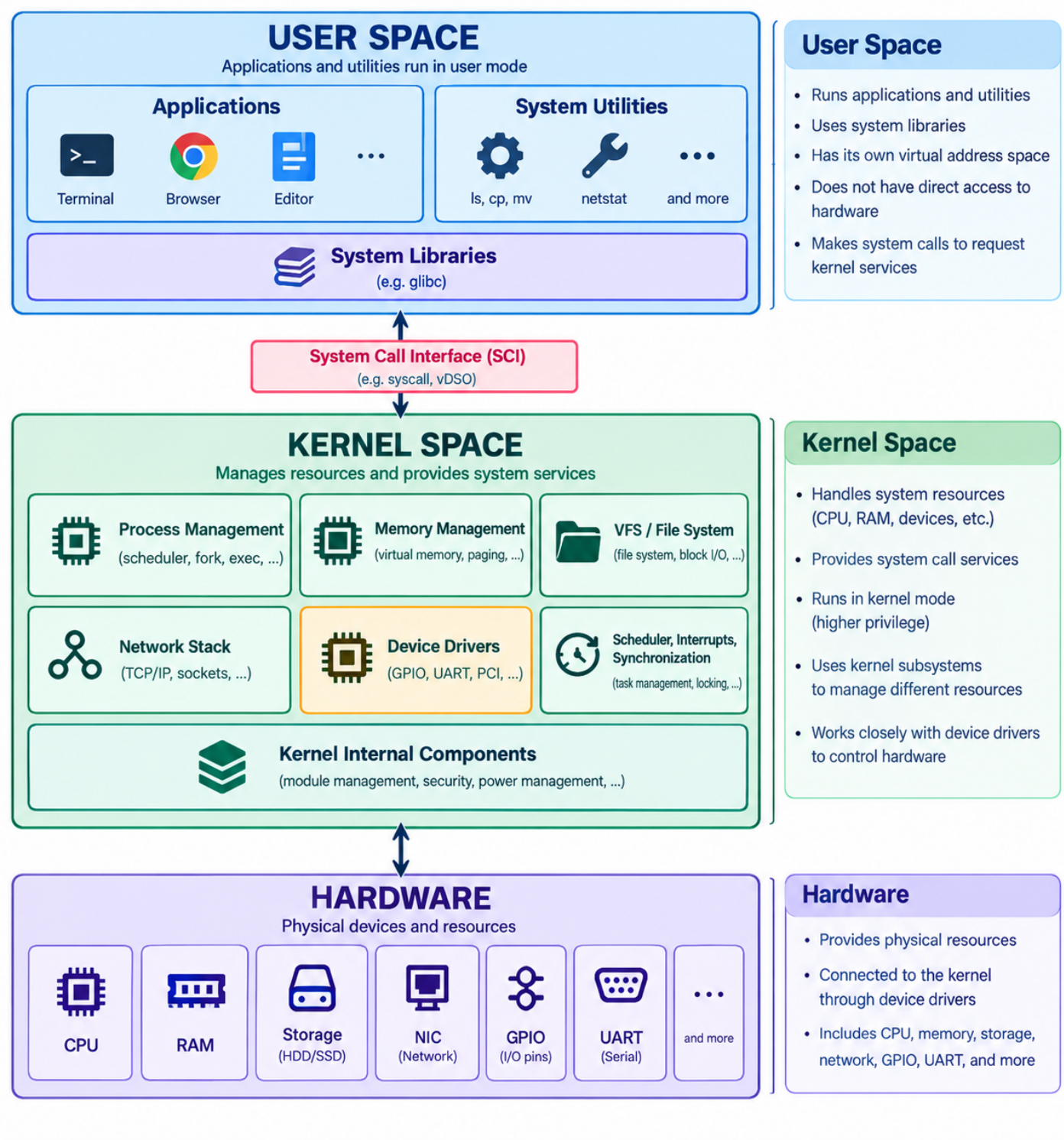
modprobe (Smart Module Probe):

- This is the "smart" version and the most commonly used one on a daily basis.
- It not only loads the .ko file but also automatically reads the dependency database (modules.dep), loads necessary supporting modules first, and then loads your main module. When removing, it also supports automatically unloading modules that are no longer in use.

Command	Purpose
`uname -r`	Show the version of the running kernel
`lsmod`	List the loaded modules
`insmod ./hello_module.ko`	Load a module from a file
`rmmod hello_module`	Remove a module by name

Command	Purpose
`modprobe module_name`	Load a module and handle its dependencies
`modprobe -r module_name`	Remove a module using modprobe
`modinfo hello_module.ko`	Show information about a module
`modinfo -F depends hello_module.ko`	Show the declared module dependencies
`dmesg`	View the kernel log
`depmod`	Create or update the module dependency database
`make`	Build the module using the Makefile
`make clean`	Remove files generated during the build process

Summary



NOTE

Question: What if one module registers to use pins 5 and 6, and then I create another module that also uses pins 5 and 6? Will the .ko file be successfully created?

- Answer: The .ko file is successfully created!

-Compiling into a .ko file (the module binary file) is just translating C source code into machine code based on kernel headers. The compiler (gcc and make) does not check if the physical hardware is already claimed, so the myled.ko file is created normally.

-However, problems happen when you load the module into the system (insmod):

- If the first module already owns the GPIO pins (via gpio_request or gpiod_get), the second module's request for those pins will be rejected, and the init function (_init) will return an error (such as -EBUSY).
- As a result, the sudo insmod command for the second module will fail and it will not load into the kernel.

Revision #2
Created 2026-10-13 10:15:36 UTC by Dat Nguyen
Updated 2026-10-14 10:38:11 UTC by Dat Nguyen